

is easy, care has to be taken in deciding the parameters for *fade* and *setpts* filters so that the videos are overlaid at the proper time. The command given below creates *video.avi* by stitching *img.avi* and the previously used *video1.avi* and *video2.avi*.

Also, note that in this case, *video1.avi* is both being faded in (in the beginning) and faded out (at the end).

```
ffmpeg -i img.avi -i video1.avi -i video2.avi -f lavfi -i
color=black -filter_complex "[0:v]fade=t=out:st=4:d=1:alpha=1,se
tpts=PTS-STARTPTS[v0];[1:v]fade=t=in:st=0:d=1:alpha=1,fade=t=ou
t:st=4:d=1:alpha=1,setpts=PTS-STARTPTS+5/TB[v1];[2:v]fade=t=in:
st=0:d=1:alpha=1,setpts=PTS-STARTPTS+10/TB[v2];[3:v]scale=1920x1
080,trim=duration=20[base];[base][v0]overlay[base1];[base1][v1]
overlay[base2];[base2][v2]overlay[outv]" -map [outv] video.avi
```

As you would have guessed by now, a script can be used to construct the above command. Once we have the list of videos, the sequence they need to be stitched in, and the lengths of the videos, a script can speed up this process and make it less error prone. The base video stream (represented as *[base]*) can also be generated using the *nullsrc* filter.

Overlay (PIP) videos

We used the *overlay* filter as a background stream and then concatenated multiple videos in previous examples. Overlaying can do much more. The example below shows how we can create a PIP video using the *overlay* filter:

```
ffmpeg -i img.avi -i video1.avi -filter_complex "[0:0]
scale=320:240[v0];[1:0][v0]overlay=overlay_w:overlay_h [outv]"
-map [outv] pip_video.avi
```

Here, we scale the *img.avi* to a smaller size (320 x 240) and then overlay it on top of *video1.avi*.

First, note that I am using *[0:0]* instead of *[0:v]* to refer to the video stream. (The 0th stream is generally the video stream. This notation is intuitive when there is more than just one stream of a type, for example, multiple audio streams.) The scale filter scales the video stream from first input to a smaller size. The *overlay* filter accepts two parameters that give the position of the overlaid video on top of the base video. We have used *overlay_w* and *overlay_h*, which represent the width and height of the overlaid video in FFmpeg (so the PIP video is placed at 320 x 240 pixels from the top left corner). We can also give absolute values for this parameter.

Audio

Until now, we have looked at the video streams only. How good is a video without its audio?

Audio is, by default, picked from the audio stream of the first input. So, in any of the commands used till now, if we did not explicitly map the *[outv]* video stream to the output file, the audio stream from the first input is

available in the output file.

For example:

```
ffmpeg -i video1.avi -i video2.avi -filter_complex "[0:0]
scale=320:240[a];[1:0][a]overlay=overlay_w:overlay_h" out.avi
```

This is similar to the *overlay* command used before. But now, the audio from the first video is available in *out.avi*.

To explicitly concatenate the audio from two files, the following command can be used:

```
ffmpeg -i video1.avi -i video2.avi -f lavfi -i color=black
-filter_complex "[0:v]fade=t=out:st=5:d=1:alpha=1,setpts
=PTS-STARTPTS[v0];[1:v]fade=t=in:st=0:d=1:alpha=1,setp
ts=PTS-STARTPTS+5/TB[v1];[2:v]scale=1920x1080,trim=du
ration=15[over];[over][va0]overlay[over1];[over1][va1]
overlay[outv];[0:a]atrim=end=5,asetpts=PTS-STARTPTS[a1];[1:a]
atrim=end=10,asetpts=PTS-STARTPTS[a2];[a1][a2]
concat=n=2:v=0:a=1[outa]" -map [outa] -map [outv] video.avi
```

Let us look at the filter graphs used for audio.

```
[0:a]atrim=end=5,asetpts=PTS-STARTPTS[a1];[1:a]
atrim=end=10,asetpts=PTS-STARTPTS[a2]
```

The audio stream from the first input is passed through two filters – *atrim* and *asetpts*. The former is used to trim the audio stream to 5s and *asetpts* sets the time stamp for the audio frames. The same filters are applied to the audio stream from the second input. Note that *atrim* is used to trim the length of the audio clip from an input. Since we are using the entire length, it can be omitted in this case.

```
[a1][a2]concat=n=2:v=0:a=1[outa]
```

The two streams (*[a1]* and *[a2]*) are not concatenated using the *concat* filter to create the final *[outa]* stream that is written to the output file.

The FFmpeg documentation is very comprehensive and can be used as a good reference while using the tool. There are lots of other options that can be used with FFmpeg to modify audio and video files as desired. 

References

- <https://www.ffmpeg.org/ffmpeg.html>
- <http://en.wikipedia.org/wiki/FFmpeg>

By: Raghu Sesha Iyengar

The author works with AgreeYa Mobility as a senior engineering manager. He has over 10 years of experience working on mobile software, especially Linux device drivers for Android devices. He can be reached at raghu.sesha@gmail.com or raghu.iyengar@agreeyamobility.net